

Robot Documentation

Alexander Greus, Kenneth Thompson, Dylan Zemlin, Spencer Smith

October 29, 2025

1 Reactive Architecture

1.1 Chosen Architecture

We kept the same general architecture (Subsumption) as project 1 except removing most reactive layers. Each layer is placed into a list, with earlier/higher layer able to override and inform lower/later layers. Again, this architecture was a good choice, since we can separate all the required behaviors/tasks into individual layers. As well, in this project higher level behaviors inform the decisions of lower level behaviors, so we again can mirror that with our priority list data structure. As well, we are glad to have chosen this architecture in project 1, as we were able to simply add and adjust behaviors very easily for project 2. Though we do not implement every layer as described in the Subsumption paper, we do implement something equivalent to the 'plan changes', 'monitor changes', and the 'avoid objects' layers.

1.2 Architecture Code - High Level Overview

The implementation of subsumption can be found in a few different places

- **Behavior Priority List** In the main spin loop, a list of behaviors is checked in the order inserted, that is higher priority behaviors are earlier in the list. The first behavior that changes any type of robot state, e.g. velocity, heading, plan state, takes control of that loop iteration and will prevent any further behaviors from triggering. For example the NavigatorBehavior ensures the robot is making progress towards its currently desired waypoint by submitting velocities and headings, and if does so lower behaviors do not run for that iteration.
- **Object Oriented Behaviors** Each behavior is represented as a class (TaskManagerBehavior, NavigatorBehavior, PathFollowerBehavior), inheriting from a base Behavior class. This design makes it incredibly simple to implement the chosen architecture as we can easily put a list of behaviors together that can be ran using the same virtual function (behavior->run()).
- **Sensor Data** Each behavior has access to a context object that contains the latest sensor reading and global state information such as whether or not the robot is escaping, etc.

1.3 Data Structures

1.3.1 Bot

The bot is our main data structure, which is just a class, with members including the Context, which as discussed before has sensor data and any other pieces of shared state, any required publishers and subscribers, as well as our priority list of behaviors and pending tasks. The behavior priority list and pending tasks are simply arrays of their respective element types, that being objects inheriting from the Behavior base class and `std::string` respectively.

1.3.2 Task Representation

- **PointFt** Just 2 floats to represent a Cartesian coordinate, (x,y).
- **Task** 2 PointFts the first representing the start and the last representing the end point of a task. Also 2 booleans for tracking whether either have been visited.
- **Waypoint** This is what we actually deal with when deciding where to go, generated by the task manager. Includes the point, the task id from which this point came from, and a boolean for checking if we have started going here.

2 Algorithms and Behaviors

The behaviors as listed below are in order from highest priority to lowest priority (e.g. Task Manager is highest).

2.1 Task Manager

This behavior checks if the bot already has a generated task plan for visiting waypoints. If we do already have a plan we early exit and do nothing, as we only need a new plan when the current one has been completed. If we do need to generate a new plan, we go through our algorithm to find the best path. First we loop through the tasks we have parsed through our custom message topic, and add them to an array or 'remaining' or unvisited waypoints. Then we loop through these 'remaining' nodes. On the first iteration we check what the closest node is to our current actual location, and add that as the first waypoint in our plan, while removing it from our 'remaining' array. On subsequent iterations, we use the most recently added node's location as our 'current' location to check for the closest to that location, and so on, continuing until the 'remaining' array of nodes is empty. Thus we construct a mostly optimal path, minimizing distance between individual waypoints in our plan.

2.2 Navigator

The navigator behavior uses the plan generated by our Task Manager, and determines the current heading and velocity. We first check if we have made progress since the last iteration towards our current goal waypoint. Next we check if we've made it within one foot of our

goal, if so we remove that waypoint. After we also check if that was the final waypoint, resetting our plan and signalling we are ready to generate a new one. If we haven't made any progress, we remove that node, and if that was the start of a task, we also remove the task's end point too. Finally we calculate our heading and velocity based according to the current position and desired waypoint.

2.3 Path Follower

The path follower behavior is a stub for now which simply calls the navigator behavior, we added this in case we needed further functionality in the future.